EduAssist AI: LLMOps Report

API Cost, Latency and Prompt Version Management | MLOps Experiential Learning Assignment-2

1. LLMOps and how it differs from MLOps

MLOps manages datasets, feature pipelines, training, deployment and monitoring of models. LLMOps extends this to the whole LLM application, where prompts, model versions, retrieved context and output guardrails each change independently and shift quality. It adds inference economics (cost follows tokens; latency follows output length) and non-determinism, which needs automated checks such as JSON validity. Prompts therefore become versioned, evaluated and promoted production artifacts.

2. Setup

No paid API was available, so, as the assignment allows, the model is Qwen/Qwen2.5-1.5B-Instruct, hosted locally on a 4-core, 15 GB, CPU-only VM (Python 3.12.9, torch 2.15.0, transformers 5.22.1, MLflow 3.17.2), with greedy decoding (temperature 0) and a 256-token output cap. Tokens come from the model's tokenizer (input counts include chat-template tokens). Cost is estimated from a published hosting price for the same model: 0.10 USD per million input and output tokens (USD; retrieved 31 October 2026; source page updated 21 July 2026).

3. Task 1 and Task 2: latency, tokens and cost

Question	Latency (s)	Input tokens	Output tokens	Cost (USD)
Q1	116.13	47	218	0.00002650
Q2	103.42	49	174	0.00002230
Q3	120.52	48	234	0.00002820
Q4	127.31	47	256	0.00003030
Q5	112.96	47	223	0.00002700

Prompt v1 was sent for each question; all five requests succeeded. Latency averaged 116.07 s (median 116.13, minimum 103.42, maximum 127.31, standard deviation 8.88). The slowest request was Q4 at 127.31 s: it generated 256 output tokens, the cap, and latency tracks output length closely (r = 0.97, about 0.5 s per token) because a CPU generates one token at a time. Repeating v1 in Task 3 gave identical token counts but a 107.93 s average, so latency varies by about 7% between runs.

Average cost was 0.00002686 USD per request, a projected 0.2686 USD for 10,000 requests a month. Output tokens make up 82% of it, so the recommendation is to shorten responses: ask for concise answers and cap output length. Capping at 128 tokens would cut the monthly estimate to about 0.1756 USD (35% less) but can truncate answers, so quality must be checked. Task 3 agrees: concise v3 costs 16% less per request than v1.

4. Task 3: prompt versioning and MLflow evaluation

In experiment eduassist_llmops_assignment, one registry prompt (eduassist_qa_prompt) holds version 1 (basic), 2 (structured) and 3 (JSON output), each with a commit message. Each response also received a boolean valid_json feedback.

Prompt Version	Avg Latency	Avg Input Tokens	Avg Output Tokens	Avg Cost (USD)	Valid JSON Rate	Recommended?
v1	107.93 s	47.6	221.0	0.00002686	0%	No
v2	92.71 s	69.6	193.8	0.00002634	0%	No
v3	63.69 s	103.6	121.0	0.00002246	0%	Yes

Promotion justification. I promoted v3 to production, conditionally. **JSON validity:** no version passed the strict check (0% for all three). All five v3 responses were wrapped in a markdown code fence despite the instruction; with the fence removed, 5 of 5 parsed and matched the schema, while the prose of v1 and v2 stayed at 0%. **Quality:** v3 answers were concise and mostly accurate, but Q4 described bias and variance backwards with high confidence, so that field is unreliable; v1 on Q4 and v2 on Q3 hit the output cap. **Latency:** v3 was fastest (63.69 s against 92.71 and 107.93). **Tokens:** it produced the fewest output tokens (121.0 against 193.8 and 221.0) despite the longest prompt (103.6 input tokens). **Cost:** it was cheapest (0.00002246 USD per request). The rule was highest valid_json, then schema compliance, then lowest latency; with validity tied at 0%, latency decided, and the alias is confirmed in the MLflow registry. The promotion is conditional: v3 needs a fence-tolerant parser or constrained decoding, and a re-test on more questions.

5. Limitations

Five questions and one run per prompt mean differences, especially in latency, include noise. A 1.5B model on a CPU is far slower than a hosted API, and costs are estimates, not invoices.